1. Introduction

Confused-101 is a custom programming language designed and implemented from scratch as part of a Compiler Design course. The language is built using Flex for lexical analysis and Bison for parsing. It executes programs through an Abstract Syntax Tree (AST) interpreter and also generates Three-Address Code (TAC) as an intermediate representation.

The goal was to build a complete compilation pipeline — from raw source text all the way through lexing, parsing, AST construction, semantic execution, and intermediate code generation — using a language with its own unique syntax and vocabulary.

2. Tools and Implementation

- Flex — lexical analyser, tokenises the source file
- Bison — parser, builds the AST from the token stream
- C — the implementation language for the AST, symbol table, and interpreter
- MinGW / GCC on Windows — compilation toolchain

Build command:

```
bison -d confused.y
flex confused.l
gcc confused.tab.c lex.yy.c -o confused.exe -lm
```

Run command:

```
confused.exe input.txt output.txt
```

3. Compiler Pipeline

The compiler processes a Confused-101 source file through five stages:

#	Stage	What happens
1	Lexical Analysis	Flex scans the source and produces tokens — keywords, identifiers, literals, operators
2	Parsing	Bison matches tokens to grammar rules and constructs the AST
3	Symbol Table	Variables and functions are stored in a frame-based call stack as the AST is walked
4	Execution	The AST is walked recursively — execute() handles statements, eval() handles expressions
5	TAC Generation	A second AST walk by gen_stmt() and gen() prints Three-Address Code instructions

4. Features Unique to Confused-101

The following features either do not exist in standard C or work very differently. These are the original contributions of this language.

Feature	Confused-101	Why unique
Readable operators	`a add b / a mul b`	Arithmetic uses English words instead of symbols — no ambiguity with pointer operators
Word comparisons	`x grt y / x lrt y`	grt, lrt, eql, neql are clearer than > < == != for beginners
Mfun entry point	`Mfun { ... }`	Single keyword replaces int main(void) — cleaner program structure
# terminator	`In x @ 10#`	# ends every statement instead of ; — visually distinct, harder to confuse with other syntax
$ separator	`func(a $ b $ c)`	$ separates arguments instead of , — no conflict with printf format strings
@ assignment	`x @ 5#`	@ is assignment — = is never used, eliminating the classic = vs == bug entirely
Inc() with trace	`Inc(x)#`	Increments x and automatically prints a trace — built-in debugging feedback
flp range loop	`flp(0:10) { }`	Clean start:end range syntax — no init/condition/step to write, auto-detects inc or dec direction
Linked-list stack	`stk mystack#`	Stack is an unlimited linked-list — C has no built-in stack type at all
Linked-list queue	`line myqueue#`	Queue is an unlimited linked-list — C has no built-in queue type at all
incident/check/nop	`incident(x) { check 1: }`	Switch-case with unique keywords — nop replaces default, no fallthrough by design
bool type	`bool flag @ true#`	Native bool with true/false literals — C requires stdbool.h or manual 0/1
Exp() power	`Exp(2 $ 8)`	Power operator as a first-class language feature — C requires calling pow() manually
String variables	`ch name @ "hello"#`	ch doubles as a string type — assign and print full strings, not just single chars
strjoin / takepart	`strjoin(a $ b)`	String concatenation and substring as built-in operators — C needs strcat/strncpy manually
MY_ARRAY syntax	`MY_ARRAY In scores(5)#`	Array declaration is a distinct statement — size is explicit and bounds are checked at runtime
Ask() input	`Ask(x)#`	Single-keyword user input — C requires printf + scanf + format string
TAC output	`automatic on every run`	Every program automatically prints its own Three-Address Code — no separate compiler pass needed

5. Sample Program

The following program exercises most features of the language in one place:

```
Mfun {
    In x @ 10#
    flt y @ 3.14#
    bool flag @ true#
    ch word @ "hello"#

    // arithmetic
    In pw @ Exp(2 $ 8)#
    Inc(x)#

    // conditional
    Iff(x grt 5) {
        Show("x is big")#
    }

    // range loop
    flp(0:3) {
        Show(i)#
    }

    // switch
    incident(x) {
        check 11 : { Show("eleven")# }
        nop        { Show("other")# }
    }

    // stack
    stk mystack#
    lastadd(mystack $ 42)#
    lastdrag(mystack)#

    // array
    MY_ARRAY In scores(3)#
    scores[0] @ 95#
    Show(scores[0])#

    // strings
    Show(strjoin(word $ " world"))#
    Show(strlen(word))#
    Show(takepart(word $ 1 $ 3))#
}
```

6. What Was Implemented

Confused-101	Standard C
`In x @ 10#`	`int x = 10;`
`flt y @ 3.14#`	`float y = 3.14;`
`bool flag @ true#`	`#include<stdbool.h>  bool flag = true;`
`ch word @ "hello"#`	`char* word = "hello";`
`x @@ 5#`	`x += 5;`
`Iff / oriff / or`	`if / else if / else`
`wlp(x lrt 5) { }`	`while(x < 5) { }`
`flp(0:10) { }`	`for(int i=0; i<10; i++) { }`
`incident / check / nop`	`switch / case / default`
`In f(In a $ In b) { }`	`int f(int a, int b) { }`
`Ask(x)#`	`printf("Enter: "); scanf("%d",&x);`
`Show(x)#`	`printf("%d\n", x);`
`stk s# / line q#`	`no built-in stack/queue in C`
`MY_ARRAY In arr(5)#`	`int arr[5];`
`strjoin(a $ b)`	`strcat(a, b)`
`strlen(a)`	`strlen(a)`
`takepart(a $ 1 $ 3)`	`strncpy(dest, a+1, 3)`
`Exp(2 $ 8)`	`pow(2, 8)`
`Inc(x)#`	`x++;  // no trace in C`
`TAC printed automatically`	`requires separate compiler pass`

7. Conclusion

Confused-101 successfully implements a complete compiler pipeline for a custom programming language. All planned features were built and verified, including several additions made during development such as bool type, linked-list stacks and queues, switch-case, arrays, and string operations.

The most distinctive aspect of the project is that the language is not just a thin wrapper over C syntax — it introduces genuinely different constructs: readable word-based operators, a range loop that auto-detects direction, unlimited linked-list data structures, built-in trace output on Inc(), and automatic TAC generation on every run. These make Confused-101 a language with its own character rather than a renamed version of C.